

Trilha Frontend Engineering - Básico ao Sênior

Plano de estudos técnico, progressivo e aplicável para evolução de básico a sênior

HTML	CSS	JavaScript	TypeScript	React
Next.js	Arquitetura	Performance	Segurança	Acessibilidade

Documento formatado a partir do conteúdo anexado pelo usuário

Gerado em 06/06/2026

Como usar esta trilha

ORIENTAÇÃO DE USO

Este PDF organiza a trilha em camadas progressivas: fundamentos da Web, engenharia de aplicações, UI Engineering e senioridade técnica. A leitura pode ser feita sequencialmente ou por seção, usando o sumário como navegação.

Sumário

Como usar esta trilha	2
1. Objetivo da trilha	3
2. Níveis da trilha	3
3. Fundamentos da Web	5
4. HTML - do básico ao sênior	6
5. CSS - do básico ao sênior	8
6. JavaScript - do básico ao sênior	11
7. TypeScript - do básico ao sênior	13
8. React - do básico ao sênior	15
9. Next.js - do básico ao sênior	17
10. Outros frameworks e quando usar	20
11. Estado, dados e arquitetura de aplicação	21
12. Design System e UI Engineering	23
13. Acessibilidade	24
14. Performance Web	25
15. Testes Frontend	27
16. Segurança Frontend	28
17. Arquitetura Frontend	30
18. Build, tooling e DX	31
19. Observabilidade Frontend	32
20. Conteúdo específico por framework	33
21. Projetos práticos obrigatórios	35
22. Checklist de nível sênior	37
23. Ordem ideal de estudo	38
24. Resumo final da trilha	38

1. Objetivo da trilha

Formar um desenvolvedor frontend capaz de:

- Criar interfaces acessíveis, responsivas e performáticas.
- Dominar HTML, CSS, JavaScript e TypeScript em profundidade.
- Trabalhar com React, Next.js e outros frameworks modernos.
- Projetar arquitetura frontend escalável.
- Criar design systems e componentes reutilizáveis.
- Testar aplicações com qualidade profissional.
- Integrar frontend com APIs REST, GraphQL, BFFs e backends reais.
- Diagnosticar problemas de performance, estado, renderização, acessibilidade e segurança.
- Atuar como frontend pleno/sênior em produtos reais.

2. Níveis da trilha

Nível básico

Foco: entender a Web e criar páginas funcionais.

Conteúdos:

- HTML semântico.
- CSS básico.
- Layout responsivo.
- JavaScript básico.
- DOM.
- Eventos.
- Formulários.
- Fetch API.
- Git básico.
- Deploy simples.

Resultado esperado:

- Criar páginas estáticas e pequenas interações.
- Entender como navegador interpreta HTML, CSS e JavaScript.
- Consumir uma API simples.

Nível intermediário

Foco: construir aplicações modernas.

Conteúdos:

- JavaScript assíncrono.
- TypeScript.
- React.
- Componentização.
- Gerenciamento de estado.
- Rotas.

- Formulários.
- Validação.
- Testes básicos.
- Consumo de APIs.
- CSS modularizado.
- Tailwind ou CSS-in-JS.
- Acessibilidade inicial.
- Performance inicial.

Resultado esperado:

- Criar SPAs e aplicações frontend completas.
- Usar TypeScript com segurança.
- Separar componentes, hooks, serviços e páginas.
- Criar UI responsiva e reutilizável.

Nível avançado

Foco: arquitetura, qualidade, performance e escala.

Conteúdos:

- React avançado.
- Next.js App Router.
- Server Components.
- SSR, SSG, ISR e streaming.
- Design systems.
- Monorepo.
- Testes E2E.
- Storybook.
- Performance Web.
- Core Web Vitals.
- Acessibilidade WCAG.
- Segurança frontend.
- Observabilidade.
- Microfrontends conceitual.
- Arquitetura de estado.
- Cache client/server.

Resultado esperado:

- Criar aplicações robustas de produção.
- Definir padrões de arquitetura frontend.
- Medir e otimizar performance.
- Criar componentes reutilizáveis e documentados.
- Trabalhar com times maiores.

Nível sênior

Foco: decisão técnica, escala, produto e liderança.

Conteúdos:

- Arquitetura frontend corporativa.
- Design system escalável.
- Governança de componentes.
- Performance budget.
- Acessibilidade como requisito não funcional.
- Segurança por padrão.
- Frontend observability.
- Edge rendering.
- BFF.
- Server-driven UI.
- Estratégia de migração.
- Estratégia de testes.
- Code review.
- Mentoria.
- RFCs e ADRs.
- Trade-offs entre React, Next.js, Vue, Angular, Svelte, Astro e outras abordagens.

Resultado esperado:

- Liderar decisões técnicas de frontend.
- Diagnosticar problemas complexos.
- Criar padrões de engenharia.
- Orientar times.
- Escolher tecnologia por contexto, não por moda.

3. Fundamentos da Web

O que estudar

Antes de qualquer framework, o desenvolvedor precisa entender a plataforma Web:

- Browser.
- HTTP/HTTPS.
- DNS básico.
- URL.
- Request/response.
- HTML.
- CSS.
- JavaScript.
- DOM.
- CSSOM.
- Eventos.
- Renderização.
- Acessibilidade.
- Segurança.
- Performance.

O DOM representa a estrutura do documento carregado no navegador como uma árvore de objetos, permitindo que scripts manipulem conteúdo, estrutura e estilo da página.

O que dominar para nível sênior

Um frontend sênior deve saber explicar:

- O que acontece quando o usuário acessa uma URL.
- Como o navegador baixa HTML, CSS, JS, imagens e fontes.
- Como o DOM é construído.
- Como CSS afeta layout e renderização.
- Como JavaScript bloqueia ou interage com a renderização.
- Como cache HTTP influencia performance.
- Como cookies, CORS e headers impactam segurança.
- Como SSR, CSR, SSG e streaming mudam a experiência do usuário.

4. HTML - do básico ao sênior

Nível básico

Estudar:

- Estrutura de um documento HTML.
- `<!DOCTYPE html>`.
- `<html>`, `<head>`, `<body>`.
- `<title>`.
- Meta tags.
- Headings.
- Parágrafos.
- Links.
- Imagens.
- Listas.
- Tabelas.
- Formulários.
- Inputs.
- Buttons.

REFERÊNCIA TÉCNICA

A MDN mantém uma referência completa dos elementos HTML agrupados por função, o que deve ser usado como base de consulta permanente.

Nível intermediário

Estudar HTML semântico:

- `<header>`.
- `<nav>`.
- `<main>`.
- `<section>`.

- <article>.
- <aside>.
- <footer>.
- <figure>.
- <figcaption>.
- <time>.
- <address>.
- <details>.
- <summary>.
- <dialog>.

Também estudar:

- Formulários acessíveis.
- label.
- fieldset.
- legend.
- required.
- autocomplete.
- inputmode.
- aria-describedby.
- Validação nativa.
- Tabelas com caption, thead, tbody, th, scope.

REFERÊNCIA TÉCNICA

A recomendação prática da MDN é usar HTML semântico, ou seja, "o elemento certo para o trabalho certo", porque o navegador já fornece vários recursos de acessibilidade nativos quando os elementos corretos são usados.

Nível avançado

Estudar:

- SEO técnico com HTML.
- Open Graph.
- Twitter Cards.
- Meta robots.
- Canonical URL.
- Structured data.
- JSON-LD.
- Lazy loading de imagens.
- srcset.
- sizes.
- Imagens responsivas.
- Preload.
- Prefetch.
- Preconnect.
- Acessibilidade nativa.

- Foco.
- Navegação por teclado.
- Landmarks.
- Ordem lógica do documento.

Nível sênior

Dominar:

- HTML como contrato semântico da interface.
- Acessibilidade sem excesso de ARIA.
- SEO técnico em aplicações renderizadas no servidor.
- HTML performático.
- Estratégia de documentos para SSR/SSG.
- Estrutura semântica para leitores de tela.
- Formulários grandes e acessíveis.
- Componentes que preservam semântica mesmo dentro de frameworks.

Critérios de domínio:

- Saber revisar uma página e identificar problemas semânticos.
- Saber substituir div e span desnecessários por elementos corretos.
- Saber criar formulários usáveis por teclado e leitores de tela.
- Saber explicar quando usar button e quando usar a.
- Saber criar HTML que funcione antes do JavaScript carregar.

5. CSS - do básico ao sênior

Nível básico

Estudar:

- Seletores.
- Propriedades.
- Valores.
- Classes.
- IDs.
- Box model.
- Margin.
- Padding.
- Border.
- Width.
- Height.
- Display.
- Position.
- Colors.
- Fonts.
- Backgrounds.
- Pseudo-classes.

- Pseudo-elements.

CONCEITO IMPORTANTE

A propriedade display define se um elemento se comporta como bloco, inline ou se cria layouts como flex e grid para seus filhos.

Nível intermediário

Estudar layout moderno:

- Flexbox.
- CSS Grid.
- Positioning.
- Responsividade.
- Media queries.
- Mobile-first.
- Fluid typography.
- CSS variables.
- Transitions.
- Animations.
- Transform.
- Z-index.
- Stacking context.
- Overflow.
- Object-fit.
- Aspect-ratio.

CONCEITO IMPORTANTE

Custom properties, também chamadas de variáveis CSS, permitem reutilizar valores no documento e são acessadas por var().

Nível avançado

Estudar:

- Cascade.
- Specificity.
- Inheritance.
- Cascade layers.
- Container queries.
- Logical properties.
- CSS nesting.
- CSS modules.
- Design tokens.
- Theme system.
- Dark mode.
- Reduced motion.

- Fluid layout.
- Clamp.
- Min/max.
- Viewport units modernas.
- Scroll snapping.
- Subgrid.
- Anchor positioning, quando aplicável.
- Estratégias de arquitetura CSS.

CONCEITO IMPORTANTE

A especificidade é o peso usado pelo navegador dentro do algoritmo de cascata para decidir qual declaração CSS será aplicada a um elemento. Cascade layers ajudam a organizar a precedência de estilos em bases grandes, reduzindo disputas de especificidade entre folhas de estilo. Container queries permitem aplicar estilos com base no tamanho ou características do contêiner, e não apenas no viewport.

Nível sênior

Dominar:

- Arquitetura CSS escalável.
- Design tokens.
- Temas.
- Component variants.
- CSS performático.
- Estratégias anti-especificidade.
- Criação de design system.
- Integração com Figma/tokens.
- Acessibilidade visual.
- Contraste.
- Estados de foco.
- Estados de erro.
- Motion design responsável.
- CSS para SSR.
- CSS crítico.
- Redução de layout shift.
- Estratégia entre Tailwind, CSS Modules, Sass, CSS-in-JS e vanilla CSS.

REFERÊNCIA TÉCNICA

Tailwind é um framework utility-first que permite compor interfaces diretamente no markup com classes utilitárias.

Critérios de domínio:

- Saber criar uma arquitetura CSS que aguente crescimento.
- Saber evitar cascata accidental.
- Saber padronizar espaçamentos, cores, tipografia e breakpoints.

- Saber criar componentes responsivos sem depender apenas de media query global.
- Saber diagnosticar bug de layout, overflow, stacking context e especificidade.

6. JavaScript - do básico ao sênior

Nível básico

Estudar:

- Variáveis.
- Tipos primitivos.
- Objetos.
- Arrays.
- Operadores.
- Condicionais.
- Loops.
- Funções.
- Escopo.
- Hoisting.
- Closures básicos.
- DOM.
- Eventos.
- Manipulação de classes.
- Manipulação de atributos.
- Fetch básico.

REFERÊNCIA TÉCNICA

A MDN define o guia de JavaScript como uma visão geral da linguagem e direciona a referência para detalhes exaustivos de cada recurso.

Nível intermediário

Estudar:

- ES Modules.
- Destructuring.
- Spread/rest.
- Template literals.
- Arrow functions.
- Array methods.
- Map.
- Set.
- WeakMap.
- WeakSet.
- Date.
- Intl.
- JSON.

- Promises.
- Async/await.
- Error handling.
- Fetch API.
- AbortController.
- LocalStorage.
- SessionStorage.
- Cookies.
- Event delegation.

REFERÊNCIA TÉCNICA

A Fetch API fornece uma interface moderna para buscar recursos pela rede e substitui o XMLHttpRequest com uma API mais poderosa e flexível.

Nível avançado

Estudar:

- Execution context.
- Call stack.
- Heap.
- Event loop.
- Task queue.
- Microtask queue.
- Promises internals.
- Closures avançados.
- Prototypes.
- this.
- Classes.
- Inheritance.
- Composition.
- Modules.
- Tree shaking.
- Memory leaks.
- Debounce.
- Throttle.
- Generators.
- Iterators.
- Proxy.
- Reflect.
- Web Workers.
- Service Workers.
- Performance API.
- IntersectionObserver.
- ResizeObserver.
- MutationObserver.

REFERÊNCIA TÉCNICA

A MDN descreve o modelo de execução JavaScript como a infraestrutura abstrata do runtime da linguagem, incluindo como a execução é organizada pelo ambiente.

Nível sênior

Dominar:

- Modelo mental profundo de runtime.
- Assincronismo e concorrência no navegador.
- Cancelamento de requests.
- Race conditions.
- Memory leaks.
- Event delegation em escala.
- Web Workers para tarefas pesadas.
- Service Workers para PWA/offline/cache.
- Otimização de bundle.
- Uso correto de APIs nativas.
- Debugging avançado no DevTools.
- Profiling de CPU e memória.
- Estratégia de erro e observabilidade no client.

CONCEITO IMPORTANTE

Service workers atuam como proxies entre aplicação, navegador e rede, permitindo experiências offline, interceptação de requests, atualização de assets, push notifications e background sync.

Critérios de domínio:

- Saber explicar event loop com microtasks e macrotasks.
- Saber identificar vazamento de memória.
- Saber cancelar request quando componente desmonta.
- Saber mover processamento pesado para Web Worker.
- Saber reduzir JavaScript enviado ao browser.
- Saber escrever código resiliente a falhas de rede.

7. TypeScript - do básico ao sênior

Nível básico

Estudar:

- Tipos primitivos.
- Arrays.
- Objects.
- Functions.
- Type aliases.
- Interfaces.

- Optional properties.
- Union types.
- Literal types.
- Type inference.
- Type annotations.

CONCEITO IMPORTANTE

TypeScript é uma linguagem fortemente tipada construída sobre JavaScript, com foco em melhor tooling e escala de aplicações.

Nível intermediário

Estudar:

- Generics.
- Type narrowing.
- Discriminated unions.
- Intersection types.
- Utility types.
- Partial.
- Pick.
- Omit.
- Record.
- ReturnType.
- Parameters.
- Readonly.
- Enums vs union literals.
- Type guards.
- Assertion functions.
- unknown vs any.
- never.

Nível avançado

Estudar:

- Conditional types.
- Mapped types.
- Template literal types.
- Recursive types.
- Branded types.
- satisfies.
- Const assertions.
- Module augmentation.
- Declaration files.
- Type-safe API clients.
- Zod + TypeScript.
- Runtime validation.

- Type inference from schemas.
- Type-safe routes.
- Type-safe forms.
- Type-level constraints.

Nível sênior

Dominar:

- TypeScript como ferramenta de arquitetura.
- Modelagem de domínio com tipos.
- Evitar any.
- Usar unknown corretamente.
- Criar SDKs tipados.
- Criar design systems tipados.
- Tipar contratos de API.
- Tipar estados impossíveis.
- Criar tipos que guiam o uso correto da aplicação.
- Integrar validação runtime com tipagem estática.
- Configurar tsconfig rigoroso.
- Reduzir complexidade de tipos quando prejudica legibilidade.

Critérios de domínio:

- Saber modelar um fluxo complexo com discriminated unions.
- Saber impedir estados inválidos em tempo de compilação.
- Saber criar tipos reutilizáveis sem exagerar em abstração.
- Saber revisar PRs TypeScript considerando legibilidade e segurança.

8. React - do básico ao sênior

Nível básico

Estudar:

- JSX.
- Components.
- Props.
- State.
- Events.
- Conditional rendering.
- Lists.
- Keys.
- Forms.
- Controlled components.
- Component composition.

Nível intermediário

Estudar:

- Hooks.
- useState.
- useEffect.
- useMemo.
- useCallback.
- useRef.
- useReducer.
- Context API.
- Custom hooks.
- Error boundaries.
- React Router.
- Form libraries.
- Data fetching.
- State management.

CONCEITO IMPORTANTE

React Hooks permitem usar recursos do React dentro de componentes e também criar hooks próprios combinando hooks internos.

Nível avançado

Estudar:

- Render cycle.
- Reconciliation.
- Component identity.
- Keys avançado.
- Controlled vs uncontrolled.
- Derived state.
- Composition patterns.
- Compound components.
- Render props.
- Context performance.
- Suspense.
- Lazy loading.
- Concurrent rendering.
- Server Components.
- Memoization.
- React Profiler.
- Hydration.
- Streaming.
- Server Actions, quando usando frameworks compatíveis.
- State machines.
- Headless components.

CONCEITO IMPORTANTE

React Server Components são um tipo de componente renderizado antecipadamente em ambiente separado do client app ou do servidor SSR tradicional, podendo rodar em build time ou por request.

Nível sênior

Dominar:

- Separação entre componente de UI, componente de composição e lógica de domínio.
- Estratégia de estado local, global, server state e URL state.
- Performance real, sem memoização prematura.
- Estratégia de renderização.
- Component API design.
- Design system em React.
- SSR/hydration bugs.
- Suspense boundaries.
- Error boundaries.
- Testabilidade.
- Acessibilidade de componentes.
- Migração entre versões.
- Code splitting.
- Bundle analysis.
- Estratégia de forms em escala.

REFERÊNCIA TÉCNICA

A própria documentação do React alerta que `useMemo` deve ser usado apenas como otimização de performance; se o código depende dele para funcionar corretamente, o problema real deve ser corrigido primeiro.

Critérios de domínio:

- Saber identificar render desnecessário.
- Saber explicar quando usar Context, Zustand, Redux, TanStack Query ou URL state.
- Saber criar componente reutilizável sem acoplamento excessivo.
- Saber lidar com estados de loading, erro, vazio, sucesso e revalidação.
- Saber criar UI resiliente a falhas de API.

9. Next.js - do básico ao sênior

Nível básico

Estudar:

- App Router.
- Páginas.
- Layouts.
- Rotas.

- Links.
- Metadata.
- Server Components.
- Client Components.
- Static assets.
- CSS modules.
- Deploy.

CONCEITO IMPORTANTE

Next.js possui App Router e Pages Router; o App Router é o modelo mais novo e suporta recursos modernos do React, como Server Components.

Nível intermediário

Estudar:

- Data fetching.
- SSR.
- SSG.
- ISR.
- Dynamic routes.
- Route handlers.
- Middleware.
- Loading UI.
- Error UI.
- Not found.
- Forms.
- Authentication.
- Cookies.
- Headers.
- Redirects.
- API routes/route handlers.

CONCEITO IMPORTANTE

No App Router, layouts e páginas são Server Components por padrão, permitindo buscar dados e renderizar partes da UI no servidor; Client Components entram quando há interatividade ou uso de APIs do navegador.

Nível avançado

Estudar:

- Streaming.
- Suspense.
- Cache.
- Revalidation.
- Server Actions.

- Partial prerendering, quando disponível no contexto do projeto.
- Edge runtime.
- Middleware avançado.
- Image optimization.
- Font optimization.
- SEO técnico.
- Internationalization.
- Auth em SSR.
- BFF com Next.
- Segurança de cookies.
- Observabilidade.
- Performance.

REFERÊNCIA TÉCNICA

Next.js documenta fetching em Server Components usando operações assíncronas como fetch ou ORM/database, além de streaming para partes da UI que dependem de dados não cacheados.

Nível sênior

Dominar:

- Estratégia entre SSR, SSG, ISR e CSR.
- Cache no servidor e no cliente.
- Revalidação por tempo e por evento.
- Segurança em cookies e headers.
- Autenticação com sessão segura.
- Separação entre frontend, BFF e backend.
- Evitar vazamento de segredo para client bundle.
- Medição de performance.
- Observabilidade.
- SEO avançado.
- Multi-tenant frontend.
- Migração Pages Router -> App Router.
- Design system dentro de monorepo.
- Deploy em Vercel, AWS, containers ou Kubernetes.

REFERÊNCIA TÉCNICA

Next.js trata caching como técnica para armazenar resultado de data fetching e computações para servir respostas futuras mais rapidamente.

Critérios de domínio:

- Saber decidir se uma página deve ser estática, dinâmica, server-rendered ou client-rendered.
- Saber debugar problema de hidratação.
- Saber evitar waterfall de dados.
- Saber proteger rotas e dados sensíveis.
- Saber medir impacto real no Core Web Vitals.

10. Outros frameworks e quando usar

Vue e Nuxt

Vue é uma alternativa forte para aplicações progressivas, dashboards e produtos com curva de aprendizado amigável. A Composition API permite criar estado reativo, computed state e watchers usando funções importadas como `ref()` e `reactive()`. Nuxt é um framework full-stack baseado em Vue com roteamento por arquivos, auto-imports e SSR configurado por padrão.

Usar quando:

- O time já conhece Vue.
- O produto precisa de boa produtividade.
- O projeto se beneficia de simplicidade e estrutura clara.
- Há necessidade de SSR/SSG com ecossistema Vue.

Angular

Angular é mais opinativo e comum em ambientes corporativos. Versões atuais usam componentes standalone por padrão, reduzindo a dependência de NgModules em novos projetos.

Usar quando:

- O time quer framework completo e padronizado.
- O produto é enterprise.
- Há necessidade de arquitetura rígida.
- O time valoriza DI, forms robustos, roteamento e convenções oficiais.

Dominar para nível sênior:

- Components.
- Signals.
- RxJS.
- Services.
- Dependency Injection.
- Reactive Forms.
- Guards.
- Interceptors.
- Lazy loading.
- Standalone components.
- Change detection.
- Angular testing.

Svelte e SvelteKit

Svelte usa compilação para gerar componentes que fazem pouco trabalho no navegador, enquanto SvelteKit é o framework para aplicações robustas e performáticas com Svelte.

Usar quando:

- Performance e simplicidade são prioridades.
- O time aceita uma abordagem menos dominante que React.
- O projeto se beneficia de menor boilerplate.

- A aplicação precisa ser leve.

Astro

Astro é otimizado para sites orientados a conteúdo e renderiza a maior parte da página como HTML estático, adicionando ilhas de JavaScript apenas onde há interatividade.

Usar quando:

- Blog.
- Documentação.
- Landing pages.
- Marketing sites.
- Portfólios.
- Conteúdo com alta exigência de performance.
- Pouca interatividade global.

Vite

Vite é uma ferramenta moderna de build frontend com suporte a frameworks por plugins e templates para diferentes stacks.

Dominar:

- Dev server.
- HMR.
- Build.
- Plugins.
- Environment variables.
- Code splitting.
- Library mode.
- Monorepo usage.
- Integração com React, Vue, Svelte e vanilla TS.

11. Estado, dados e arquitetura de aplicação

Nível básico

Estudar:

- Estado local.
- Props.
- Eventos.
- Formulários simples.
- Fetch em componente.

Nível intermediário

Estudar:

- Lifting state up.
- Context.

- Custom hooks.
- Server state.
- URL state.
- Form state.
- Cache.
- Loading/error states.
- TanStack Query.
- Zustand.
- Redux Toolkit.
- React Hook Form.
- Zod.

Nível avançado

Estudar:

- Normalização de estado.
- Cache invalidation.
- Optimistic updates.
- Retry.
- Pagination.
- Infinite scroll.
- Debounced search.
- URL como fonte de verdade.
- State machines.
- XState.
- Offline state.
- Conflict resolution.
- Real-time updates.
- WebSocket.
- SSE.

Nível sênior

Dominar:

- Separar estado de UI, estado de servidor, estado de formulário e estado de rota.
- Evitar estado global desnecessário.
- Criar arquitetura previsível.
- Projetar cache client-side.
- Tratar concorrência e race conditions.
- Definir política de revalidação.
- Criar contratos de API tipados.
- Alinhar frontend com backend/BFF.

Critérios de domínio:

- Saber explicar por que nem todo estado deve ir para Redux/Zustand.
- Saber usar URL para filtros, paginação e busca.

- Saber lidar com atualização otimista e rollback.
- Saber criar experiência resiliente com API instável.

12. Design System e UI Engineering

Conteúdos essenciais

Estudar:

- Design tokens.
- Component library.
- Accessibility-first components.
- Variants.
- Slots.
- Composition.
- Theming.
- Dark mode.
- Responsividade.

Estados:

- default.
- hover.
- focus.
- active.
- disabled.
- loading.
- error.
- success.
- empty.
- Documentação.
- Versionamento.
- Breaking changes.
- Visual regression.
- Storybook.
- Figma handoff.
- Tokens pipeline.

REFERÊNCIA TÉCNICA

Storybook é uma ferramenta para construir, testar e documentar componentes e páginas de UI de forma isolada, sem precisar executar a aplicação inteira.

Nível sênior

Dominar:

- Criar componentes headless.
- Criar componentes compostos.

- Definir API de componente.
- Evitar props explosivas.
- Projetar acessibilidade desde o componente base.
- Versionar design system.
- Documentar uso correto.
- Garantir consistência visual em vários produtos.
- Criar governança de contribuição.

Entregáveis recomendados:

- Button.
- Input.
- Select.
- Checkbox.
- Radio.
- Dialog.
- Dropdown.
- Tabs.
- Tooltip.
- Toast.
- Table.
- DataTable.
- Pagination.
- FormField.
- Card.
- Sidebar.
- Navbar.
- Command menu.
- Empty state.
- Error state.
- Skeleton.

13. Acessibilidade

Fundamentos

Estudar:

- WCAG.
- Semântica.
- Contraste.
- Foco visível.
- Teclado.
- Leitores de tela.
- Labels.
- Alt text.
- Landmarks.

- ARIA.
- Estados dinâmicos.
- Motion reduction.
- Form errors.
- Skip links.

REFERÊNCIA TÉCNICA

WCAG 2.2 reúne recomendações para tornar conteúdo Web mais acessível, e a W3C recomenda usar a versão mais recente por ser compatível com versões anteriores da WCAG 2.x.

Nível sênior

Dominar:

- Acessibilidade como requisito de arquitetura.
- Testes com teclado.
- Testes com leitor de tela.
- Auditoria com axe.
- Navegação por foco.
- Dialogs acessíveis.
- Combobox acessível.
- Tabelas acessíveis.
- Formulários acessíveis.
- Anúncios dinâmicos com live regions.
- Evitar ARIA incorreto.
- Criar checklist de PR acessível.

Critérios de domínio:

- Toda funcionalidade precisa funcionar sem mouse.
- Todo input precisa ter label.
- Todo erro precisa ser identificado e anunciado.
- Todo componente interativo precisa ter foco visível.
- A semântica nativa deve ser priorizada antes de ARIA.

14. Performance Web

Fundamentos

Estudar:

- Core Web Vitals.
- LCP.
- CLS.
- INP.
- TTFB.
- FCP.
- Bundle size.

- Code splitting.
- Lazy loading.
- Image optimization.
- Font optimization.
- Critical CSS.
- Caching.
- CDN.
- Prefetch.
- Preload.
- Hydration.
- JavaScript cost.

CONCEITO IMPORTANTE

Core Web Vitals mede experiência real de usuário em carregamento, interatividade e estabilidade visual; o conjunto atual inclui LCP, INP e CLS.

Nível avançado

Estudar:

- Lighthouse.
- WebPageTest.
- Chrome DevTools Performance.
- Coverage.
- Network waterfall.
- Bundle analyzer.
- React Profiler.
- Long tasks.
- Main thread blocking.
- Third-party scripts.
- Resource hints.
- Edge caching.
- Streaming SSR.
- Partial hydration.
- Islands architecture.

Nível sênior

Dominar:

- Criar performance budget.
- Medir antes de otimizar.
- Otimizar imagens e fontes.
- Reduzir JavaScript.
- Evitar hidratação desnecessária.
- Controlar scripts de terceiros.
- Melhorar INP com redução de trabalho na main thread.
- Melhorar LCP com SSR, preload, imagem correta e cache.

- Melhorar CLS com dimensões estáveis.
- Criar dashboards de performance.
- Relacionar performance com conversão e SEO.

Critérios de domínio:

- Saber diagnosticar por que uma página está lenta.
- Saber diferenciar problema de rede, servidor, bundle, renderização ou terceiro.
- Saber justificar cada otimização com métrica.

15. Testes Frontend

Nível básico

Estudar:

- Teste manual estruturado.
- Unit tests.
- Assertions.
- Vitest/Jest.
- Testing Library.
- Testes de componentes simples.

REFERÊNCIA TÉCNICA

Testing Library incentiva testes que se pareçam com a forma como usuários interagem com o software, aumentando a confiança nos testes.

Nível intermediário

Estudar:

- React Testing Library.
- Queries por role.
- User events.
- Mock de APIs.
- MSW.
- Testes de formulário.
- Testes de hooks.

Testes de estados:

- loading.
- error.
- empty.
- success.

REFERÊNCIA TÉCNICA

A documentação de queries da Testing Library recomenda priorizar seletores que reflitam a interação real do usuário.

Nível avançado

Estudar:

- Playwright.
- Cypress.
- E2E.
- Component testing.
- Visual regression.
- Accessibility testing.
- Contract testing.
- Snapshot com cautela.
- Test data builders.
- Mock Service Worker.
- CI test strategy.

REFERÊNCIA TÉCNICA

Playwright permite automação confiável em Chromium, Firefox e WebKit usando uma API comum, inclusive com TypeScript.

Nível sênior

Dominar:

- Estratégia de testes por risco.
- Pirâmide de testes frontend.
- Testes críticos E2E.
- Testes de acessibilidade no CI.
- Testes visuais para design system.
- Testes de contrato com backend.
- Testes resilientes e pouco frágeis.
- Evitar testar implementação interna.
- Definir o que não testar.

Critérios de domínio:

- Saber montar pipeline com unit, component, E2E e accessibility tests.
- Saber reduzir flakiness.
- Saber usar Playwright para jornada crítica.
- Saber usar Storybook para estados de componente.
- Saber decidir entre mock e API real.

16. Segurança Frontend

Conteúdos essenciais

Estudar:

- XSS.
- CSRF.

- CORS.
- CSP.
- Clickjacking.
- Secure cookies.
- HttpOnly.
- SameSite.
- Token storage.
- LocalStorage risks.
- Session management.
- Dependency vulnerabilities.
- Supply chain.
- NPM audit.
- Secret leakage.
- Environment variables.
- File upload risks.
- Open redirects.
- DOM-based XSS.
- iframe security.
- OAuth2/OIDC no frontend.
- PKCE.
- Sanitização.
- Escape de output.

Nível sênior

Dominar:

- Nunca expor segredo no bundle.
- Preferir cookie HttpOnly para sessão quando a arquitetura permitir.
- Entender risco de tokens em LocalStorage.
- Aplicar CSP.
- Validar input no frontend sem confiar somente nele.
- Tratar HTML dinâmico com extrema cautela.
- Revisar dependências.
- Proteger rotas client e server.
- Garantir autorização no backend/BFF, não só no frontend.
- Prevenir vazamento de dados por renderização indevida.

Critérios de domínio:

- Saber revisar uma aplicação React/Next em busca de XSS e vazamento de token.
- Saber configurar headers de segurança.
- Saber diferenciar autenticação, autorização e proteção visual de rota.
- Saber explicar por que esconder botão no frontend não é autorização.

17. Arquitetura Frontend

Nível intermediário

Estudar:

- Feature-based architecture.
- Component-based architecture.
- Shared components.
- Hooks.
- Services.
- API clients.
- Types.
- Utils.
- Pages/routes.
- Layouts.

Nível avançado

Estudar:

- Clean frontend.
- Ports/adapters no frontend.
- Domain-driven frontend.
- Modular architecture.
- Monorepo.
- Package boundaries.
- Dependency rules.
- Design system package.
- API SDK package.
- BFF.
- Microfrontends conceitual.
- Module federation.
- Server-driven UI.
- Feature flags.
- Experimentation.

Nível sênior

Dominar:

- Criar arquitetura que suporte crescimento de time e produto.
- Definir boundaries claros.
- Evitar acoplamento entre features.
- Separar domínio de UI.
- Definir padrões de erro, loading e empty states.
- Criar SDK tipado para backend.
- Definir estratégia de cache.
- Definir estratégia de autenticação.

- Definir estratégia de build/deploy.
- Criar ADRs para decisões frontend.
- Planejar migração incremental.

Exemplo de estrutura sênior:

```
src/  
  app/  
    routes/  
    layouts/  
    providers/  
  features/  
    auth/  
    dashboard/  
    transactions/  
    settings/  
  shared/  
    ui/  
    hooks/  
    lib/  
    config/  
    types/  
  services/  
    api/  
    auth/  
    telemetry/  
  design-system/  
    tokens/  
    components/
```

18. Build, tooling e DX

Conteúdos

Estudar:

- Node.js.
- pnpm/npm/yarn.
- Vite.
- Next build.
- TypeScript compiler.
- ESLint.
- Prettier.
- Biome.
- Stylelint.
- Husky.
- lint-staged.
- Commitlint.
- Changesets.
- Turborepo.
- Nx.
- Monorepo.
- Environment variables.

- Bundle analyzer.
- Source maps.
- CI/CD.

Nível sênior

Dominar:

- Criar DX eficiente.
- Reduzir tempo de build.
- Reduzir tempo de teste.
- Padronizar lint/format.
- Criar pacote compartilhado.
- Versionar design system.
- Controlar breaking changes.
- Criar templates internos.
- Criar scaffolding de feature.
- Automatizar qualidade em PR.

19. Observabilidade Frontend

Conteúdos

Estudar:

- Error tracking.
- Sentry.
- Logging client-side.
- Session replay com cuidado de privacidade.
- Web Vitals reporting.
- User timing.
- Performance marks.
- Analytics.
- Funnel tracking.
- Feature flags.
- A/B testing.
- Real User Monitoring.
- Synthetic monitoring.
- Correlation ID.
- Trace propagation.

Nível sênior

Dominar:

- Saber responder quais usuários foram impactados.
- Saber ligar erro frontend com trace backend.
- Saber medir erro por versão.
- Saber medir performance por rota.

- Saber detectar regressão após deploy.
- Saber separar métrica técnica de métrica de produto.
- Saber proteger privacidade e evitar coleta indevida.

20. Conteúdo específico por framework

React + Next.js como trilha principal

Para seu contexto de projetos com Next.js, React, Tailwind e shadcn/ui, eu colocaria React + Next.js como trilha principal. Conteúdo obrigatório:

- React.
- Hooks.
- Server Components.
- Client Components.
- Suspense.
- Error boundaries.
- Forms.
- Data fetching.
- Next App Router.
- Metadata.
- SEO.
- Cache.
- Revalidation.
- Middleware.
- Auth.
- Cookies.
- Route handlers.
- Streaming.
- Image optimization.
- Deploy.
- Observability.
- Testing.

Vue + Nuxt como trilha complementar

Conteúdo:

- Vue Composition API.
- SFC.
- Reactivity.
- Watchers.
- Computed.
- Vue Router.
- Pinia.
- Nuxt SSR.
- File-based routing.

- Server routes.
- Auto imports.

Angular como trilha enterprise

Conteúdo:

- Components.
- Standalone components.
- Signals.
- Services.
- DI.
- RxJS.
- Reactive forms.
- Guards.
- Interceptors.
- Lazy loading.
- Angular Material.
- Testing.

Svelte + SvelteKit como visão moderna

Conteúdo:

- Svelte compiler.
- Stores.
- Runes.
- SvelteKit routes.
- SSR.
- Forms.
- Load functions.
- Actions.
- Progressive enhancement.

Astro para conteúdo

Conteúdo:

- Content collections.
- Markdown/MDX.
- Islands architecture.
- Static generation.
- Partial hydration.
- SEO.
- Blog/documentação.
- Integração com React/Vue/Svelte.

21. Projetos práticos obrigatórios

Projeto 1 - Landing page profissional

Objetivo:

- HTML semântico.
- CSS responsivo.
- Acessibilidade.
- SEO.
- Performance.

Requisitos:

- Navbar.
- Hero.
- Cards.
- Seções.
- Formulário.
- FAQ.
- Footer.
- Lighthouse alto.
- Sem framework.

Projeto 2 - Dashboard com React + TypeScript

Objetivo:

- React.
- TypeScript.
- Componentização.
- Estado.
- Fetch.
- Formulários.
- Testes.

Requisitos:

- Login mockado.
- Dashboard.
- Tabela.
- Filtros.
- Paginação.
- Gráficos.
- Formulário com validação.
- Estados de loading, error e empty.

Projeto 3 - App Next.js full stack

Objetivo:

- Next App Router.

- Server Components.
- Client Components.
- Auth.
- SEO.
- Cache.
- API.
- Deploy.

Requisitos:

- Blog.
- Admin.
- Autenticação.
- CRUD.
- Markdown/MDX.
- Search.
- Metadata.
- Sitemap.
- Open Graph.
- Testes E2E.

Projeto 4 - Design System

Objetivo:

- UI Engineering.
- Storybook.
- Tokens.
- Acessibilidade.
- Component API.

Requisitos:

- Tokens.
- Button.
- Input.
- Select.
- Dialog.
- Dropdown.
- Tabs.
- Toast.
- DataTable.
- Dark mode.
- Documentação.
- Testes visuais.

Projeto 5 - Frontend sênior integrado ao backend

Objetivo:

- Arquitetura real.

- BFF.
- Segurança.
- Observabilidade.
- Performance.
- CI/CD.

Requisitos:

- Next.js.
- API Java/Spring.
- Auth segura.
- Cookies HttpOnly.
- RBAC.
- Dashboard.
- RUM.
- Web Vitals.
- Playwright.
- CI/CD.
- Deploy.

22. Checklist de nível sênior

Você pode considerar a trilha frontend em nível sênior quando conseguir:

- Explicar o funcionamento do navegador.
- Criar HTML semântico e acessível.
- Construir CSS escalável com design tokens.
- Dominar JavaScript assíncrono e runtime.
- Usar TypeScript para impedir estados inválidos.
- Criar componentes React reutilizáveis, acessíveis e performáticos.
- Projetar aplicações Next.js com SSR, SSG, cache e segurança.
- Escolher framework por contexto.
- Definir arquitetura de pastas e boundaries.
- Criar design system documentado.
- Medir e otimizar Core Web Vitals.
- Criar estratégia de testes.
- Proteger tokens, rotas e dados sensíveis.
- Investigar bugs de hidratação, estado, renderização e performance.
- Fazer code review técnico.
- Criar ADRs e RFCs.
- Mentorar devs júnior/pleno.
- Comunicar trade-offs para backend, produto e design.

23. Ordem ideal de estudo

Minha recomendação de sequência:

- HTML semântico e acessibilidade básica
- CSS moderno: flex, grid, responsivo, cascade e tokens
- JavaScript profundo
- TypeScript
- DOM, Fetch, Web APIs e browser
- React
- Next.js
- Estado, forms e data fetching
- Testes
- Performance
- Segurança frontend
- Design system
- Arquitetura frontend
- Observabilidade
- Frameworks complementares: Vue, Angular, Svelte, Astro
- Projetos de portfólio com documentação técnica

24. Resumo final da trilha

A trilha frontend sênior precisa ser construída em cima de quatro camadas:

Plataforma Web

HTML, CSS, JavaScript, DOM, HTTP, browser, acessibilidade e segurança.

Engenharia de Aplicações

TypeScript, React, Next.js, estado, forms, APIs, testes e arquitetura.

UI Engineering

Design systems, tokens, componentes reutilizáveis, Storybook, acessibilidade e consistência visual.

Senioridade técnica

Performance, observabilidade, segurança, decisões arquiteturais, migrações, code review, documentação, mentoria e trade-offs.

Para o seu contexto, a combinação mais estratégica seria:

- HTML + CSS moderno + JavaScript profundo + TypeScript + React + Next.js + Tailwind/shadcn + Storybook + Playwright + Web Vitals + Acessibilidade WCAG + arquitetura frontend documentada.